

CS-1201 Object-Oriented Programming

Operator Overloading

Arbish Akram

Department of Computer Science
Government College University

A Problem Worth Solving

- With built-in types we write `a + b`, `a < b` and `cout << a` without a second thought.
- For a class we write ourselves, such as `Box` or `Fraction`, C++ has **no idea** what `+` or `<` should mean.

Without overloading

```
box3 = box1.add(box2);  
if (f1.isLessThan(f2)) { ... }  
c.print(cout);
```

With overloading

```
box3 = box1 + box2;  
if (f1 < f2) { ... }  
cout << c;
```

Operator overloading lets us define what an *existing* operator does when its operands are objects of our own class. The right column reads like ordinary mathematics, and our types behave like built-in ones.

One Symbol, Many Meanings

C++ **already** chooses an operator's meaning from the **types of its operands**:

Expression	Meaning
<code>3 + 4</code>	integer addition
<code>2.5 + 1.5</code>	floating-point addition
<code>ptr + 1</code>	pointer arithmetic
<code>s1 + s2</code>	string concatenation
<code>x << 1</code>	bitwise left shift
<code>cout << x</code>	stream insertion
<code>x >> 1</code>	bitwise right shift
<code>cin >> x</code>	stream extraction

`string` is a **class**, not a built-in type. Its `+`, `==` and `<<` were written by library authors using exactly the technique in this lecture.

<< and >>: Two Completely Different Jobs

```

1 int main() {
2     int x = 5;           // 0000 0101
3     cout << x << endl;   // 5  (insertion)
4     cout << (x << 1) << endl; // 10 (left shift)

5     cout << (x >> 1) << endl; // 2  (right
6     shift)
7     return 0;
}

```

- Left operand is a **stream**: insertion runs.
- Both operands are **ints**: the shift runs.
- The parentheses matter: `cout << x << 1` would print 5 and then 1.

x = 5	0	0	0	0	0	1	0	1
x << 1 = 10	0	0	0	0	1	0	1	0
x >> 1 = 2	0	0	0	0	0	0	1	0

The amber cell is the new 0 shifted in.
The bit pushed off the other end is discarded.

The Core Mechanism: Operators Are Functions

An overloaded operator is an ordinary function whose name is the keyword `operator` followed by the symbol, e.g. `operator+`. When an operand is of class type, the compiler **rewrites the expression** as a call to that function.

	As a member	As a non-member
<code>a + b</code>	<code>a.operator+(b)</code>	<code>operator+(a, b)</code>
<code>a < b</code>	<code>a.operator<(b)</code>	<code>operator<(a, b)</code>
<code>++a</code>	<code>a.operator++()</code>	<code>operator++(a)</code>
<code>a++</code>	<code>a.operator++(0)</code>	<code>operator++(a, 0)</code>
<code>cout << a</code>	not possible	<code>operator<<(cout, a)</code>

The rewritten call is real C++: `c.operator++()`; compiles, although nobody writes it that way.

A Counter Class: First Attempt

A Counter counts events such as clicks or visitors. The natural way to say *add one* is `++c`, exactly as with an `int`.

```

1 class Counter {
2 private:
3     unsigned int count{0};          // starts at 0
4 public:
5     unsigned int get_count() const { return count; }
6
7     void operator++() { ++count; }  // 1st attempt
8 };
9
10 int main() {
11     Counter c1, c2;
12     ++c1;                          // c1.operator++()
13     ++c2;                          // c2.operator++()
14     ++c2;
15     cout << c1.get_count() << " "
16          << c2.get_count() << endl;
17     return 0;
18 }

```

Output:

1 2

`count{0}` is a *default member initializer*: it sets the starting value without writing a constructor.

- `c1` and `c2` each own a separate count.
- `++c1` runs the function on `c1` only.

Anatomy of an Operator Function

empty parameter list: prefix form ++c

```

void operator++()
  
```

return type function name:
keyword operator + the symbol

The compiler looks at the **operand types**, just as it does when choosing among overloaded functions:

```

int n = 0;    Counter c;
++n;         // n is an int: the built-in increment runs (this can never be changed)
++c;         // c is a Counter: rewritten as c.operator++()
  
```

- Overloading is never automatic. Without `operator++` in `Counter`, the line `++c;` does not compile.
- A member operator function must be non-static: it works on the particular object it is called on.

A Hidden Problem: The void Return Type

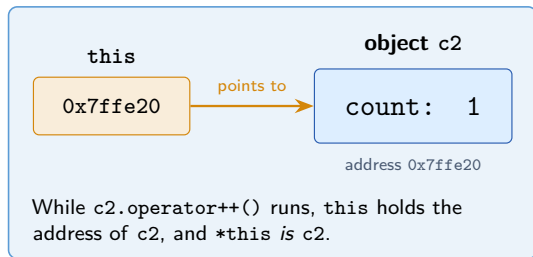
```
int n1, n2 = 0;
n1 = ++n2;      // fine: n1 = 1, n2 = 1
++(++n2);       // fine: n2 is now 3

Counter c1, c2;
c1 = ++c2;      // error: ++c2 is void
++(++c2);       // error: cannot ++ a void
```

- Our operator++() returns void, so the expression ++c2 produces **nothing**.
- That is fine as a standalone statement, but it breaks every larger expression that uses the result.
- int allows both lines. Our class should too.

++c2 should produce **the updated c2 object itself**. But how can a member function refer to *the object it was called on*? this pointer.

The this Pointer



- Inside every non-static member function, `this` is a pointer to the calling object.
- Writing `count` inside a member function is really shorthand for `this->count`.
- Dereferencing, `*this`, gives the object itself.

```
Counter& operator++() {
    ++count;           // really: ++(this->count)
    return *this;      // hand back the calling object itself
}
```

The return type `Counter&` is a *reference*: the caller receives the original object, not a copy. Now `c1 = ++c2;` compiles.

Why a Reference and Not a Copy?

```
Counter operator++() { ++count; return *this; } // Version A: returns a copy
Counter& operator++() { ++count; return *this; } // Version B: returns the object itself

Counter c;    ++(++c);    cout << c.get_count();    // what is printed?
```

Step	A: returns a copy	B: returns a reference
inner ++c	c becomes 1; returns a <i>temporary copy</i>	c becomes 1; returns c itself
outer ++	increments the temporary to 2, which is then thrown away	increments c to 2
output	1	2

With `int`, `++(++n)` adds 2. Only version B matches. **Rule:** an operator that modifies its object returns a **reference to `*this`**.

++ Has Two Forms: Prefix and Postfix

Form	What happens	Value produced
++c	increment first	the new value
c++	remember, then increment	the old value

By convention, the **postfix** version takes an extra `int` parameter that is never used. It exists only as a marker.

Both functions are named `operator++`, and neither naturally takes an argument. With identical parameter lists, how could the compiler tell them apart?

```
Counter& operator++();      // prefix: ++c   is rewritten as  c.operator++()
Counter  operator++(int);  // postfix: c++  is rewritten as  c.operator++(0)
```

- The dummy `int` is usually left unnamed; the compiler passes 0.
- The decrement operator `--` follows exactly the same convention.

A Complete Counter

```

1 class Counter {
2     unsigned int count{0};    // private by
    default
3 public:
4     unsigned int get_count() const { return count;
5     }
6
7     // Prefix ++c: increment, return *this
8     Counter& operator++() {
9         ++count;
10        return *this;
11    }
12
13    // Postfix c++: save, increment, return old
14    Counter operator++(int) {
15        Counter old = *this;    // snapshot
16        ++*this;                // reuse prefix
17        return old;
18    };

```

Tracing `c2 = c1++`; with `c1` at 1:

1. `old = *this`
⇒ `old` holds 1
2. `++*this`
⇒ `c1` becomes 2
3. `return old`
⇒ `c2` receives 1

Result: `c1 = 2`, `c2 = 1`.

With `c2 = ++c1`; instead, `c1` is incremented first and **both** end at 2.

Return Types and Their Cost

- **Prefix** returns Counter&: the object is already updated, so we hand back the object itself. No copy is made.
- **Postfix** returns Counter by value: the old state exists only in the local variable old.
- Postfix calls ++*this instead of repeating ++count. The logic lives in one place, so the two forms can never disagree.

Postfix creates an extra object and returns a copy; prefix does neither. Write ++i rather than i++ whenever you do not need the old value, e.g. in a for loop.

Never return a reference to a local variable:

```
Counter& operator++(int) { // WRONG
    Counter old = *this;
    ++*this;
    return old; // old dies here
}
```

old is destroyed when the function ends, so the caller receives a **dangling reference** to memory that no longer holds a Counter. Using it is undefined behaviour.

The General Rule for Unary Operators

```

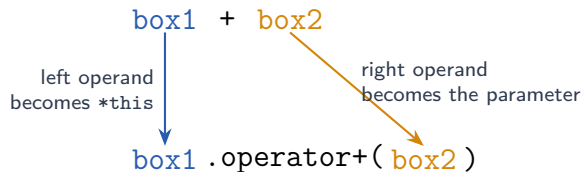
1 // Option 1: member function, NO parameters
2 class Counter {
3     unsigned int count{0};
4 public:
5     unsigned int get_count() const { return count; }
6
7     bool operator!() const { return count == 0; }
8 };
9 // !c is rewritten as c.operator!()
10
11 // Option 2: non-member function, ONE parameter
12 bool operator!(const Counter& c) {
13     return c.get_count() == 0;
14 }
15 // !c is rewritten as operator!(c)

```

Here !c asks: is the counter still at zero?

- **Member:** the single operand is the calling object, *this, so there are **no** parameters.
- **Non-member:** the operand must be passed in, so there is **one** parameter.
- Choose **one**. Defining both makes !c ambiguous.
- Option 2 only uses the public get_count(), so it needs no special access.

From One Operand to Two



- A unary member operator had **no** parameters: its only operand was the calling object.
- A binary member operator takes exactly **one** parameter, the **right** operand.
- The **left** operand is always the object the function is called on.
- Example: a `Box` class, where adding two boxes adds their matching dimensions.

Overloading operator+ for Box

```

1 class Box {
2 private:
3     double length{0.0}, breadth{0.0}, height{0.0};
4 public:
5     Box() = default;
6     Box(double l, double b, double h)
7         : length{l}, breadth{b}, height{h} { }
8
9     double getVolume() const {
10         return length * breadth * height;
11     }
12
13     // left operand: *this    right operand: rhs
14     Box operator+(const Box& rhs) const {
15         return Box{length + rhs.length,
16                     breadth + rhs.breadth,
17                     height + rhs.height};
18     }
19 };

```

- `Box()` = default; asks the compiler to generate the default constructor; the initializers make every side 0.0.
- `length{1}` is a *member initializer list* entry: it sets each side directly.
- `const Box& rhs` (*right-hand side*): no copy is made, and `rhs` cannot be changed.
- The trailing `const` promises `box1 + box2` leaves `box1` unchanged, just as `a + b` leaves `a` unchanged.
- Returns a **new** `Box` by value: the sum is neither operand.

Using operator+

```

1 int main() {
2     Box box1{1.0, 4.0, 5.0};
3     Box box2{2.0, 3.0, 5.0};
4     Box box3 = box1 + box2; // box1.operator+(
5                               box2)
6
7     cout << "box1: " << box1.getVolume() << endl;
8     cout << "box2: " << box2.getVolume() << endl;
9     cout << "box3: " << box3.getVolume() << endl;
10    return 0;
11 }

```

Output:

```

box1: 20
box2: 30
box3: 210

```

box1: $1 \times 4 \times 5 = 20$

box2: $2 \times 3 \times 5 = 30$

box3 sides:

$(1+2, 4+3, 5+5) = (3, 7, 10)$

volume $3 \times 7 \times 10 = 210$

Related Operators Are Not Automatic

Defining `+` does **not** give us `+=`: `box1 += box2`; still fails to compile. A tidy approach is to write `+=` first and build `+` on top of it.

```

1 // Inside class Box
2 Box& operator+=(const Box& rhs) {
3     length += rhs.length;
4     breadth += rhs.breadth;
5     height += rhs.height;
6     return *this;           // modified object
7 }
8
9 Box operator+(const Box& rhs) const {
10    Box result = *this;      // copy of left operand
11    result += rhs;           // reuse +=
12    return result;           // new object
13 }

```

	Changes left?	Returns
+=	yes	Box&
+	no	Box
++c	yes	Counter&
c++	(returns old)	Counter

The same pattern as prefix and postfix. The arithmetic lives only in `+=`, so the two operators can never disagree.

Where the Member Form Falls Short

Suppose we add a member function that scales a box by a number:

```
Box operator*(double k) const {
    return Box{length * k, breadth * k, height * k};
}
```

Expression	Rewritten as	
box1 * 2.0	box1.operator*(2.0)	compiles
2.0 * box1	2.0.operator*(box1) ?	a double has no member functions

Rule: a member operator only works when the **left** operand is an object of the class. For any other left operand we need a **non-member** function.

Binary Operators as Non-Member Functions

A non-member binary operator takes **two** parameters: the left operand and the right operand, in that order.

```
// Written OUTSIDE the class
Box operator*(double k, const Box& b) {
    return b * k;      // reuse the member version
}
```

- `2.0 * box1` is now rewritten as `operator*(2.0, box1)` and compiles.
- At least one parameter must be a class type, or a reference to one. That is why you can never redefine `+` for two ints.
- This function only calls a **public** member, so it needs no special access to the private data of `Box`.
- But what if a non-member operator *does* need private data? That is the job of the `friend` keyword.

Member or Non-Member?

	Member	Non-member
Unary: parameters	0	1
Binary: parameters	1	2
<code>a + b</code> becomes	<code>a.operator+(b)</code>	<code>operator+(a, b)</code>
Left operand	must be the class	any type
Private access	automatic	only if friend

How to decide:

- Left operand is **not** your class (`2.0 * box, cout << obj`): **non-member**.
- Operator **modifies** its left operand (`=, +=, ++`): **member**.

Printing and Reading Our Own Objects

In `cout << obj` the **left** operand is `cout`, an object of the library class `ostream`. This is the 2.0 * `box1` situation again: we cannot add members to `ostream` or `istream`. So `<<` and `>>` are **non-members**, and usually friends because they need private data.

```
ostream& operator<<(ostream& out, const Complex& c);    // output
istream& operator>>(istream& in, Complex& c);         // input
```

- `out` / `in` by reference: streams cannot be copied, and reading or writing changes their state.
- `const Complex&` for output: printing never modifies the object.
- `Complex&` (not `const`) for input: reading stores new values in the object.
- Both return the stream they received. We will see why in two slides.

Example: << and >> for Complex

```
class Complex {
private:
    int real{0};
    int imag{0};
public:
    Complex() = default;
    Complex(int r, int i)
        : real{r}, imag{i} { }

    friend ostream& operator<<(ostream& out,
                               const Complex& c);
    friend istream& operator>>(istream& in,
                               Complex& c);
};
```

```
ostream& operator<<(ostream& out,
                   const Complex& c) {
    out << c.real << " + " << c.imag << "i"
    ;
    return out;
}

istream& operator>>(istream& in,
                   Complex& c) {
    in >> c.real >> c.imag;
    return in;
}
```

The body writes to out, never to cout. The same function then works for files and string streams too.

Using the Overloaded Stream Operators

```
1 int main() {  
2     Complex c1;  
3     cout << "Enter the real and imaginary parts: ";  
4     cin >> c1; // operator>>(cin, c1)  
5     cout << "The complex number is " << c1 << endl; // operator<<(cout, c1)  
6     return 0;  
7 }
```

Sample run:

```
Enter the real and imaginary parts: 3  
4  
The complex number is 3 + 4i
```

The prompt is printed in main, not inside operator>>. An extraction operator should only **read**, so that it also works with files, where a prompt makes no sense.

Why Return the Stream? Cascading

```
cout << c1 << c2;

// << groups left to right:
(cout << c1) << c2;

// which the compiler turns into:
operator<<(operator<<(cout, c1), c2);
```

Returning the stream has a second benefit: we can test whether a read succeeded.

```
if (cin >> c1) { ... }
```

A stream converts to false after a failed read, e.g. when the user types letters.

1. The inner call prints c1 and **returns** cout.
2. That returned stream becomes the left operand of the outer call, which prints c2.
3. Had we returned void, step 2 would have no stream to write to, and the line would not compile.

This is called *cascading*. It works the same way for `cin >> a >> b`.

Comparing Objects

- We can now create, combine, print and read objects. The last common need is to **compare** them: dates, fractions, grades, or sorting a list of objects.
- The six comparison operators are ==, !=, <, <=, > and >=. All share one general form:

```
bool operator<op>(const ClassName& rhs) const {  
    // compare *this with rhs  
}
```

- <op> stands for any of the six operators.
- Returns bool: a comparison is true or false.
- const ClassName&: no copy, right operand unchanged.
- Trailing const: left operand unchanged.

A Fraction Class: The Constructor

```
1 class Fraction {  
2 private:  
3     int num;    // numerator  
4     int den;    // denominator, always positive  
5 public:  
6     Fraction(int n = 0, int d = 1)  
7         : num{n}, den{d} {  
8         if (den < 0) {           // move sign to num  
9             num = -num;  
10            den = -den;  
11        }  
12    }  
13    // comparison operators: next slide  
14};
```

Fraction: The Six Comparison Operators

```
// The real logic lives in == and <
bool operator==(const Fraction& rhs) const
{
    return num * rhs.den == rhs.num * den;
}
bool operator<(const Fraction& rhs) const {
    return num * rhs.den < rhs.num * den;
}
```

```
// The other four are built from them
bool operator!=(const Fraction& rhs) const
{
    return !(*this == rhs);
}
bool operator>(const Fraction& rhs) const {
    return rhs < *this;
}
bool operator<=(const Fraction& rhs) const
{
    return !(rhs < *this);
}
bool operator>=(const Fraction& rhs) const
{
    return !(*this < rhs);
}
```

Using the Comparison Operators

```

1 int main() {
2     Fraction f1{1, 2};    // 1/2
3     Fraction f2{2, 4};    // 2/4, same value as
4                             1/2
5     Fraction f3{3, 4};    // 3/4
6
7     cout << boolalpha;    // true/false, not 1/0
8     cout << "f1 == f2: " << (f1 == f2) << endl;
9     cout << "f1 != f3: " << (f1 != f3) << endl;
10    cout << "f1 <= f2: " << (f1 <= f2) << endl;
11    cout << "f3 > f1: " << (f3 > f1) << endl;
12    cout << "f3 < f1: " << (f3 < f1) << endl;
13    return 0;
14 }

```

Output:

```

f1 == f2:  true
f1 != f3:  true
f1 <= f2:  true
f3 > f1:   true
f3 < f1:   false

```

The parentheses around each comparison are required. Without them, << binds before ==, because overloading never changes precedence.

A Subtle Asymmetry: Implicit Conversion

`Fraction(int n = 0, int d = 1)` can be called with a single `int`, so it doubles as an *implicit conversion* from `int` to `Fraction`.

```
Fraction f{1, 2};
bool a = (f < 1);    // f.operator<(Fraction{1})    OK: the parameter is converted
bool b = (1 < f);    // 1.operator<(f) ?            error: 1 is an int, it has no members
```

- For a **member** operator, conversion is applied to the parameter only, never to the object on the left.
- **Classic fix:** make it a non-member friend. Then *either* argument may be converted, and both lines compile:

```
friend bool operator<(const Fraction& a, const Fraction& b);    // inside the class
```

What Can and Cannot Be Overloaded

Overloadable:

+	-	*	/	%	^
&		~	!	=	<
>	+=	-=	*=	/=	%=
^=	&=	=	<<	>>	<<=
>>=	==	!=	<=	>=	<=>
&&		++	--	,	->*
->	()	[]	new	delete	
new[]	delete[]				

Not overloadable:

- scope resolution ::
- member access .
- pointer-to-member .*
- conditional ?:
- sizeof and typeid

What Overloading Can Never Change

- **Precedence.** In $a + b * c$, $*$ still happens first.
- **Associativity.** Left-to-right for `int` stays left-to-right for our classes.
- **Arity.** A unary operator stays unary; a binary one stays binary.
- **No new operators.** You cannot invent `**` for power.
- **Built-in meaning.** At least one operand must be a user-defined type, so `+` on two `ints` always adds.

Choosing the Form and the Return Type

Member or non-member

Must be **members** `=, (), [], ->`

Must be **non-members** `<<, >>` with streams (the stream is the left operand)

Either all others; compound assignments like `+=` are usually members

Return type follows the built-in pattern

Kind	Examples	Returns
modifies object	<code>++c, +=, =</code>	<code>T& (*this)</code>
makes new value	<code>a + b, c++</code>	<code>T</code> by value
stream I/O	<code><<, >></code>	the stream by reference
comparison	<code>==, <</code>	<code>bool</code>
three-way	<code><=></code>	an ordering (<code>auto</code>)

Good Programming Practice

- **Behave like the built-in.** An overloaded `+` should add, never subtract or print.
- **Write one operator in terms of another:** `+` from `+=`, postfix from prefix, the six comparisons from `==` and `<` (or `<=>`). The logic then lives in one place.
- **Be const-correct.** Operators that do not modify an operand take it by `const&` and, as members, are marked `const`.

Summary

- An overloaded operator is a function named `operator` followed by the symbol. The compiler rewrites `a + b` as `a.operator+(b)` or `operator+(a, b)`.
- **Unary:** prefix `++` returns `*this` by reference; postfix takes a dummy `int` and returns the old value by value.
- **Binary:** a member takes one parameter (the right operand); a non-member takes two and is required when the left operand is not our class. `friend` grants private access.
- **Streams:** always non-member, usually `friend`, and always return the stream to allow cascading.
- **Comparisons:** write `==` and `<` and derive the rest, or in C++20 write `==` and `<=>`.
- Overloading never changes precedence, associativity, arity, or the meaning of operators on built-in types.